Parallel Space-Time Kernel Density Estimation

Erik Saule[†], Dinesh Panchananam[†],
Alexander Hohl[‡], Wenwu Tang[‡], Eric Delmelle[‡]

[†] Dept. of Computer Science
[‡]Dept. of Geography and Earth Sciences
UNC Charlotte
Email: {esaule,dpanchan,ahohl,wtang4,eric.delmelle}@uncc.edu

LIG seminar
(previously presented at ICPP 2017)

Outline

Space Time Kernel Density

What is it?

- Common way of visualizing events with time and place information
- Basically voxelize the space
- Give a value to each voxel that depends on the number of *neighboring* events to the voxel (with some kind of decay).
- Essentially a generalization of density maps (e.g., population density)

What is it useful for?

- Monitoring disease outbreak
- Political analysis
- Social media analysis
- Ornithology

Space-Time Kernel Density Estimate Formally

For a voxel x, y, t

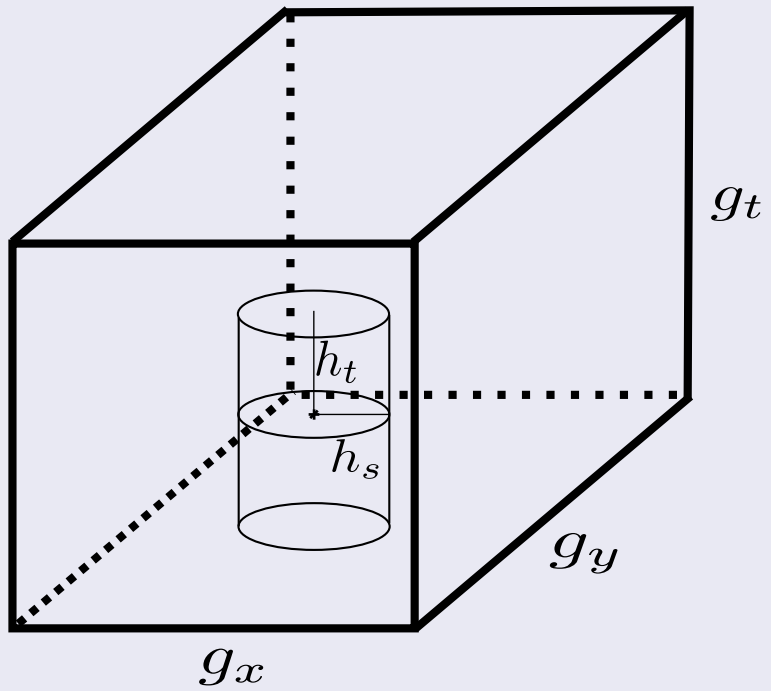
$$\hat{f}(x,y,t) = \frac{1}{nh_s^2h_t}\sum_{i|d_i<h_s,t_i<h_t} k_s\left(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s}\right)k_t\left(\frac{t-t_i}{h_t}\right)$$

$$k_s(u,v) = \frac{\pi}{2}(1-u)^2(1-v)^2$$

$$k_t(w) = \frac{3}{4}(1-w)^2$$

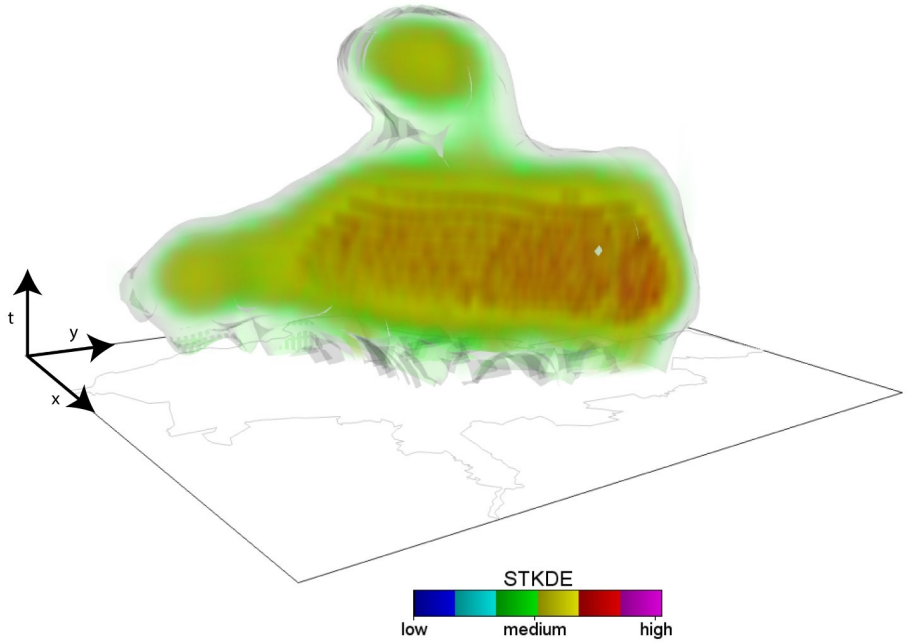
h_s is the spatial bandwidth
h_t is the temporal bandwidth
n is the number of points (events)

Each event radiates density

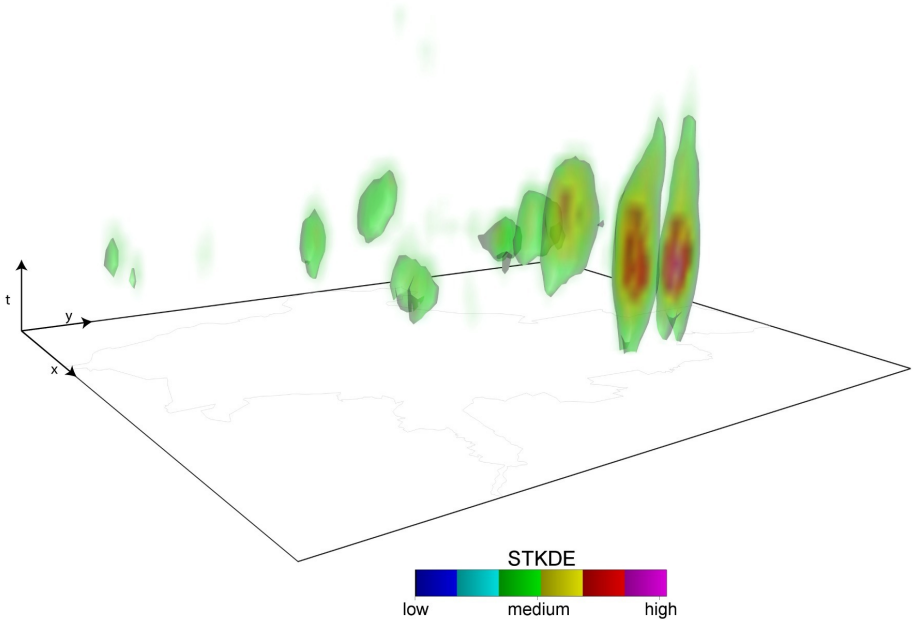


Similar to computing sums of radial basis functions which are typical in physics.

Dengue Fever in Cali, Colombia



$h_s = 2500m,\ h_t = 14days$



$h_s = 500m,\ h_t = 7days$

Outline

Voxel-Based Algorithm VB

Algorithm

for all voxels $s = (x, y, t)$ **do**
  $sum = 0$
  for all points i at x_i, y_i, t_i **do**
    if $\sqrt{(x_i - x)^2 + (y_i - y)^2} < h_s$ and $|t_i - t| \leq h_t$ **then**
      $sum+ = k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s})k_t(\frac{t-t_i}{h_t})$
  $stkde[X][Y][T] = \frac{sum}{nh_s^2 h_t}$

- $\theta(G_x G_y G_t n)$ distance tests
- $\theta(nH_s^2 H_t)$ density values
- Complexity: $\theta(G_x G_y G_t n)$

But pleasingly parallel.

Point-Based Algorithm PB

Algorithm

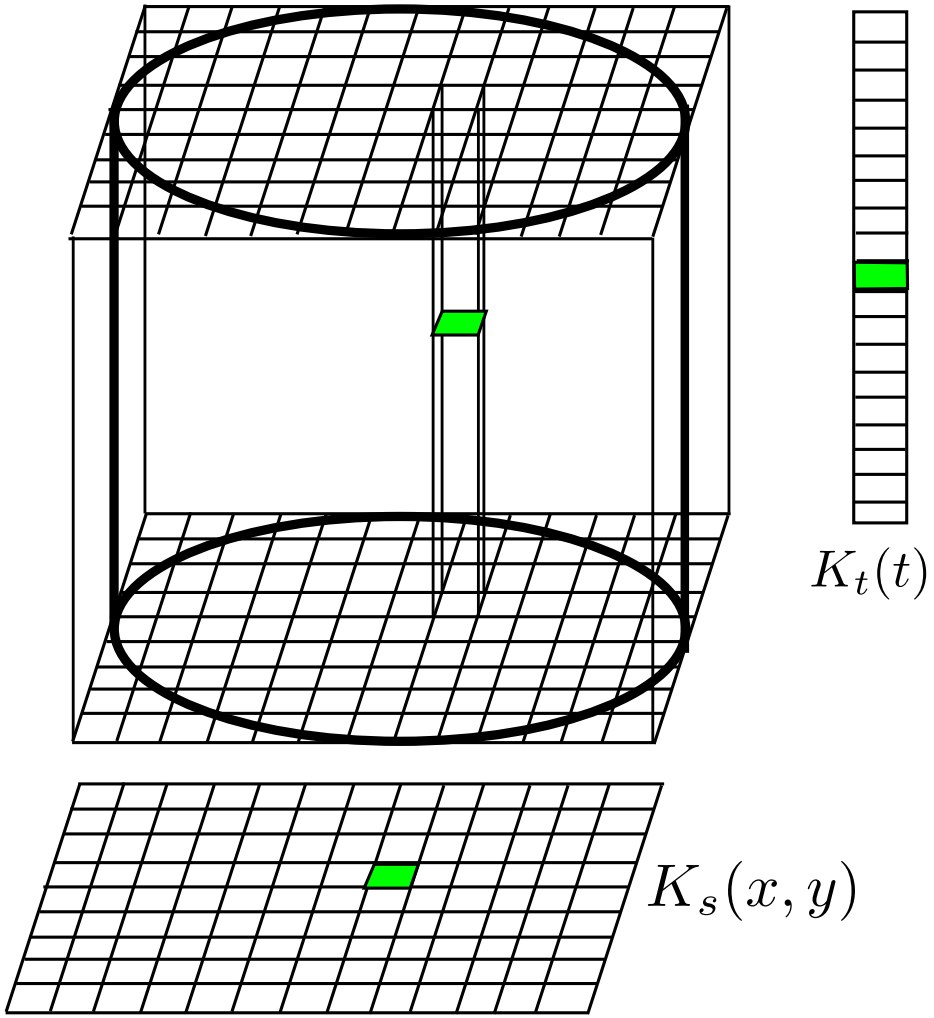
for all voxels $s = (x, y, t)$ **do**
  $stkde[X][Y][T] = 0$

for each points i at x_i, y_i, t_i **do**
  for $X_i - H_s \leq X \leq X_i + H_s$ **do**
    for $Y_i - H_s \leq Y \leq Y_i + H_s$ **do**
      for $T_i - T_s \leq T \leq T_i + H_s$ **do**
        if $\sqrt{(x_i - x)^2 + (y_i - y)^2} < h_s$ and $|t_i - t| \leq h_t$ **then**

$$stkde[X][Y][T] += \frac{k_s(\frac{x-x_i}{h_s}, \frac{y-y_i}{h_s}) k_t(\frac{t-t_i}{h_t})}{n h_s^2 h_t}$$

- $\Theta(G_x G_y G_t)$ for memory initialization
- $\Theta(n H_s^2 H_t)$ density computations
- Complexity: $\Theta(G_x G_y G_t + n H_s^2 H_t)$
- (Gain the $\theta(G_x G_y G_t n)$ distance tests from the voxel-based algorithm)

Exploiting Symmetries PB-SYM



For each point:

- Compute the K_t
- Compute the K_s
- Do the cross product

Complexity is the same, but saves computation in practice

Experimental settings

Instance	n	G_x x G_y x G_t	Size	H_s	H_t
Dengue_Lr-Lb	11056	148x194x728	79MB	3	1
Dengue_Lr-Hb	11056	148x194x728	79MB	25	1
Dengue_Hr-Lb	11056	294x386x728	315MB	2	1
Dengue_Hr-Hb	11056	294x386x728	315MB	50	1
Dengue_Hr-VHb	11056	294x386x728	315MB	50	14
PollenUS_Lr-Lb	588189	131x61x84	2MB	2	3
PollenUS_Hr-Lb	588189	651x301x84	62MB	10	3
PollenUS_Hr-Mb	588189	651x301x84	62MB	25	7
PollenUS_Hr-Hb	588189	651x301x84	62MB	50	14
PollenUS_VHr-Lb	588189	6501x3001x84	6252MB	100	3
PollenUS_VHr-VLb	588189	6501x3001x84	6252MB	50	3
Flu_Lr-Lb	31478	117x308x851	117MB	1	1
Flu_Lr-Hb	31478	117x308x851	117MB	2	3
Flu_Mr-Lb	31478	233x615x1985	1085MB	2	3
Flu_Mr-Hb	31478	233x615x1985	1085MB	4	7
Flu_Hr-Lb	31478	581x1536x5951	20260MB	5	7
Flu_Hr-Hb	31478	581x1536x5951	20260MB	10	21
eBird_Lr-Lb	291990435	357x721x2435	2391MB	2	3
eBird_Lr-Hb	291990435	357x721x2435	2391MB	6	5
eBird_Hr-Lb	291990435	1781x3601x2435	59570MB	10	3
eBird_Hr-Hb	291990435	1781x3601x2435	59570MB	30	5

Shared memory machine (A node of Copperhead):

- 2 Intel Xeon E5-2667 v3 (2 times 8 cores)
- 128GB of DRAM
- G++ 5.3 (with OpenMP 4.0)

In practice, PB-SYM is much better

Instance	Time (in seconds) VB	VB-DEC	PB	PB-DISK	PB-BAR	PB-SYM	speedup PB-SYM
Dengue_Lr-Lb	219.163	2.283	0.040	0.029	0.035	0.028	1.429
Dengue_Lr-Hb	220.591	13.878	1.298	0.564	1.152	0.499	2.601
Dengue_Hr-Lb	866.445	9.522	0.089	0.082	0.085	0.084	1.060
Dengue_Hr-Hb	871.774	55.206	5.169	2.272	4.563	2.074	2.492
Dengue_Hr-VHb	1056.172	404.845	51.885	11.478	42.994	7.431	6.982
PollenUS_Lr-Lb	518.859	7.639	1.106	0.347	0.922	0.256	4.320
PollenUS_Hr-Lb	12721.001	189.337	23.539	7.700	18.527	4.708	5.000
PollenUS_Hr-Mb	17179.482	3126.947	357.743	86.129	295.791	57.528	6.219
PollenUS_Hr-Hb			2666.104	583.175	2212.626	382.566	6.969
PollenUS_VHr-Lb			2428.126	1004.174	1949.988	759.722	3.196
PollenUS_VHr-VLb			603.789	240.236	488.388	179.834	3.357
Flu_Lr-Lb	926.360	3.691	0.035	0.032	0.034	0.032	1.094
Flu_Lr-Hb	966.328	3.797	0.081	0.046	0.070	0.042	1.929
Flu_Mr-Lb	8591.165	30.355	0.305	0.278	0.298	0.277	1.101
Flu_Mr-Hb	8957.175	32.018	0.714	0.384	0.608	0.323	2.211
Flu_Hr-Lb		536.091	5.702	5.089	5.454	5.059	1.127
Flu_Hr-Hb		591.955	12.795	6.822	10.992	7.072	1.809
eBird_Lr-Lb			396.811	147.951	322.580	125.248	3.168
eBird_Lr-Hb			6969.187	1897.051	5611.158	1067.395	6.529
eBird_Hr-Lb			8373.273	3226.016	6470.764	2229.460	3.756
eBird_Hr-Hb						34577.745	

Clearly, PB-SYM is the algorithm to make parallel.

Parallelism in STKDE is not trivial

The problem

- The algorithm is written with a **for all points** as its outer loop.
- But if the cylinder of two points intersect and the points are processed at the same time, there could be race condition in writing the `stkde` array

Naive solution

- Lock the state of cells of `stkde` to avoid the race condition
- Or use atomics when updating `stkde`
- That causes $\Theta(nH_s^2H_t)$ locks or atomics

A better solution

- Make sure intersecting cylinder are never processed at the same time.
- That is the rest of the presentation

Parallelism in STKDE is not trivial

The problem

- The algorithm is written with a **for all points** as its outer loop.
- But if the cylinder of two points intersect and the points are processed at the same time, there could be race condition in writing the `stkde` array

Naive solution

- Lock the state of cells of `stkde` to avoid the race condition
- Or use atomics when updating `stkde`
- That causes $\Theta(nH_s^2H_t)$ locks or atomics

A better solution

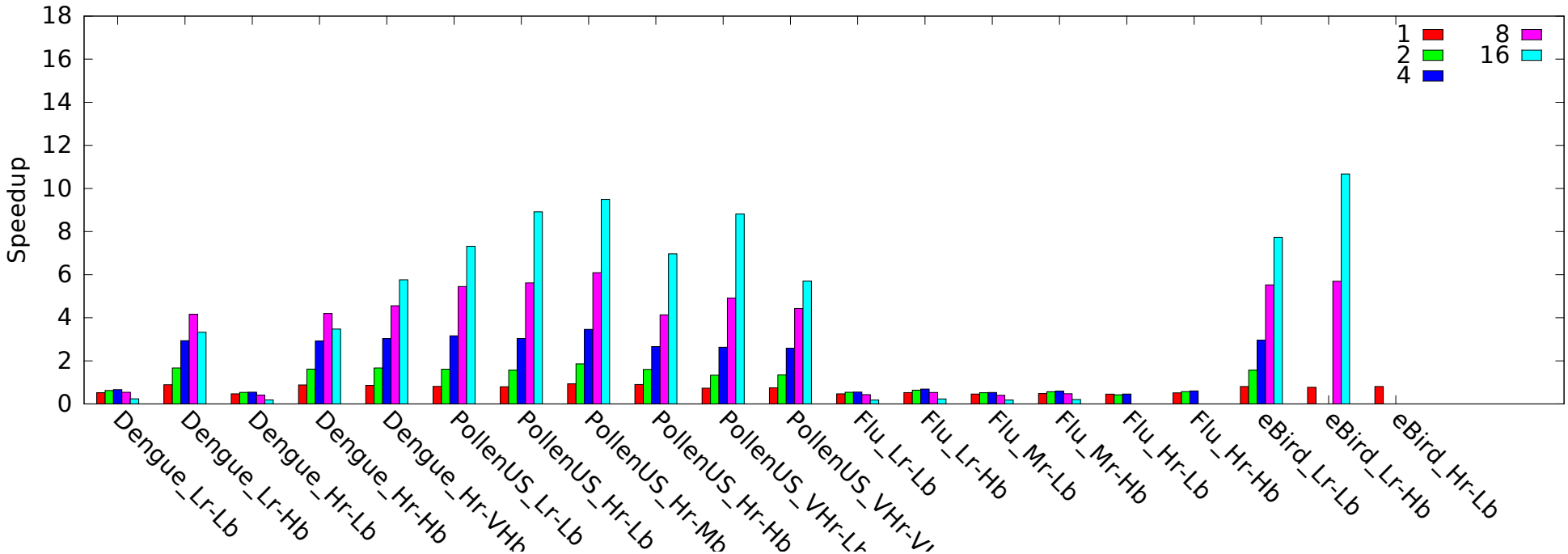
- Make sure intersecting cylinder are never processed at the same time.
- That is the rest of the presentation

Outline

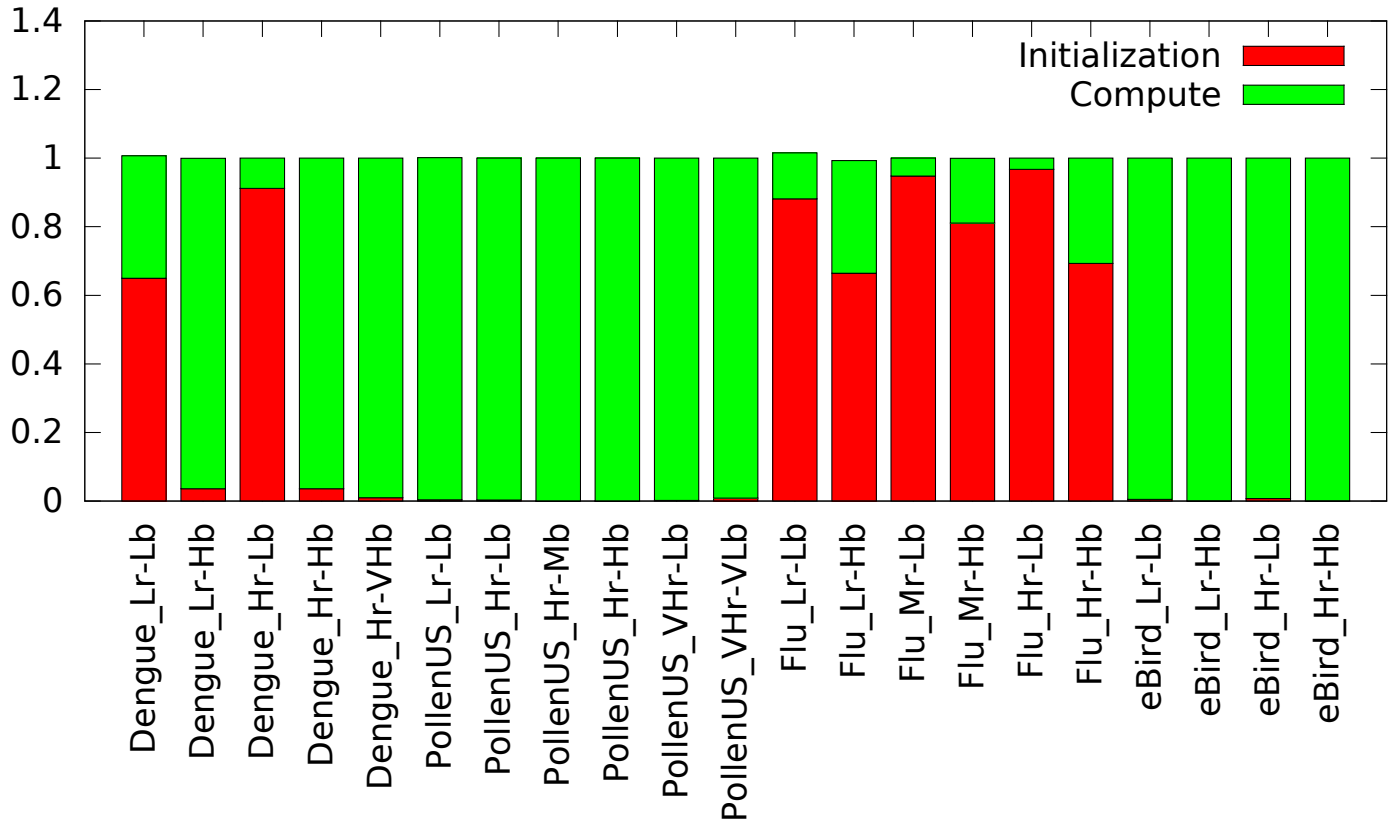
Domain Replication PB-SYM-DR

Each worker:

- Initializes its own memory buffer `stkde_local`
- Processes some points in `stkde_local` (with load balancing)
  - So no race condition
- Participates in reducing the many `stkde_local` into `stkde`



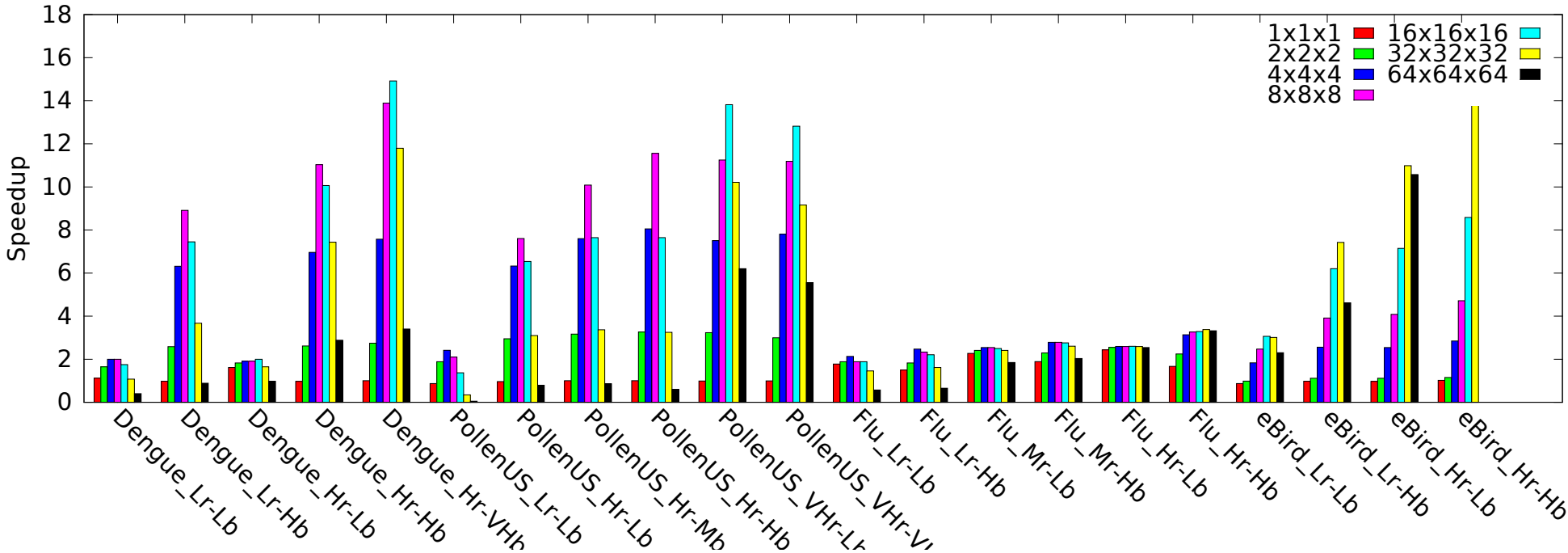
Why is DR bad? Some instances have little computation!



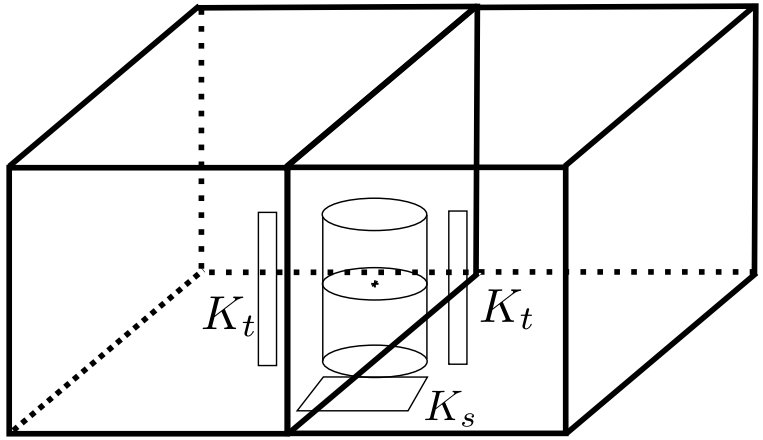
(and some run out of memory)

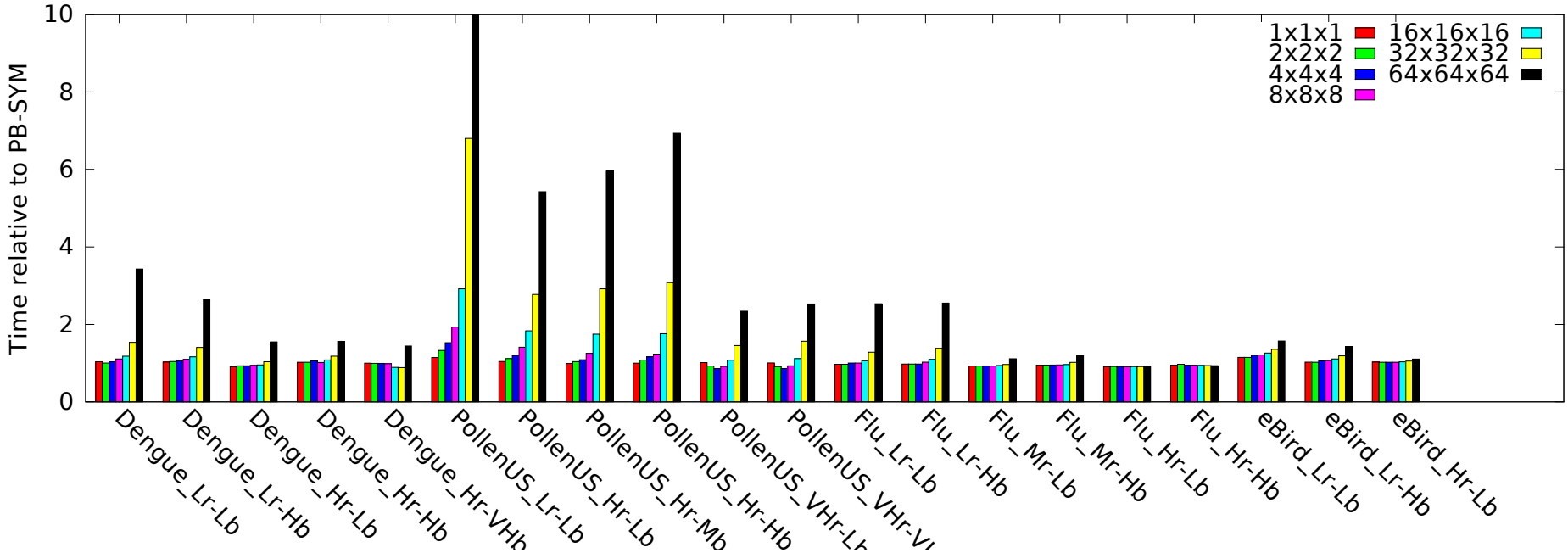
Domain Decomposition PB-SYM-DD

- Decompose the voxel domain in $K \times K \times K$ subdomains
- Each worker processes different subdomains (load balanced on the subdomains)
  - Each voxel is now processed by a unique thread, so no race condition



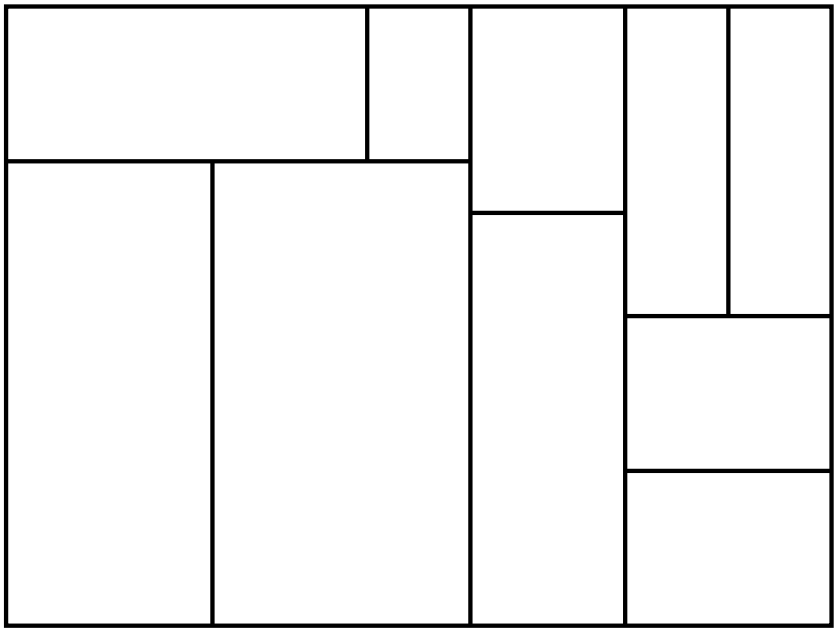
Why is DD bad? Work overhead. Some cylinders are cut!



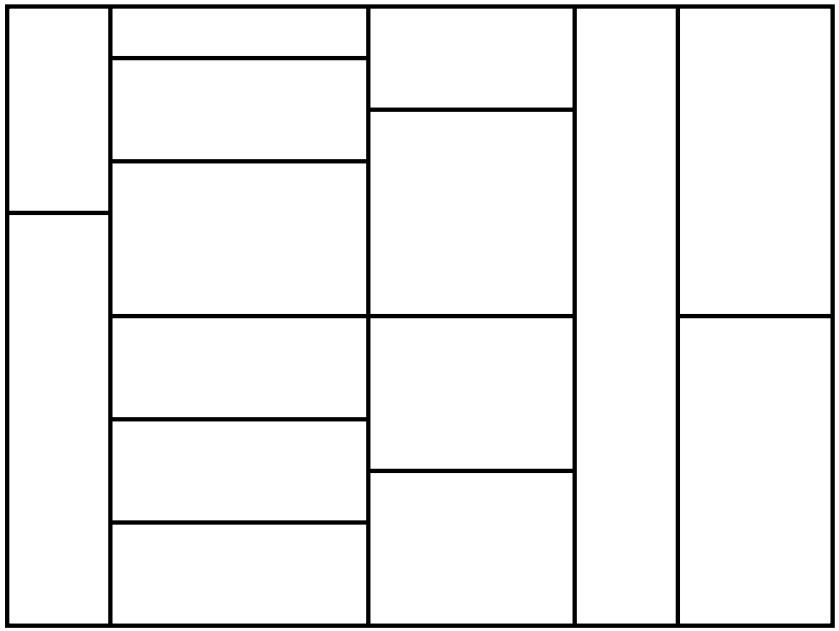


Are there better partitionings? (since ICPP17)

`PB-SYM-DD` partitions the space using a naive grid. But we can do better.



Hierarchical partitionings
(shown in 2D, but generalizes to 3D)



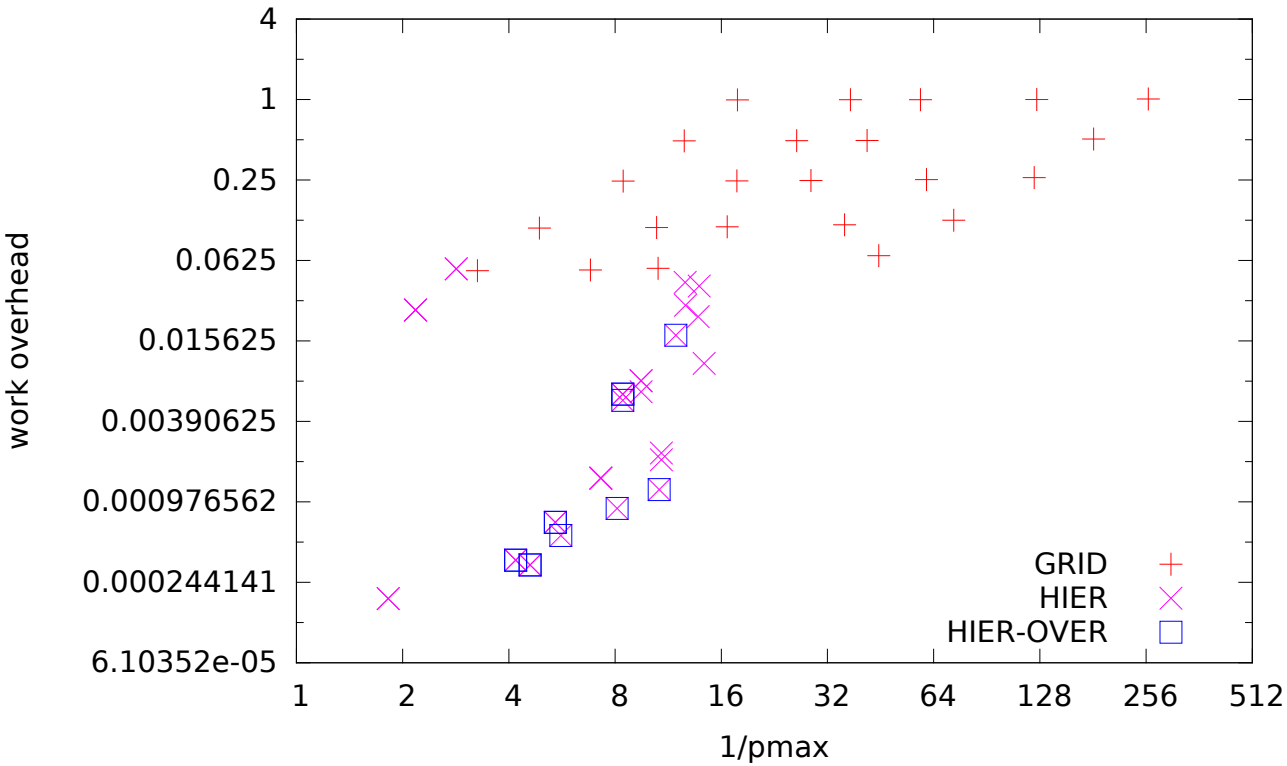
Jagged partitionings

Optimal decompositions can be found using dynamic programming

$$
\begin{aligned}
Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, P) = \min_{1 \le p < P} (\\
\min_{X_{min} < x < X_{max}} (\max & Hier(X_{min}, x, Y_{min}, Y_{max}, T_{min}, T_{max}, p), \\
& Hier(x, X_{max}, Y_{min}, Y_{max}, T_{min}, T_{max}, P - p)), \\
\min_{Y_{min} < y < Y_{max}} (\max & Hier(X_{min}, X_{max}, Y_{min}, y, T_{min}, T_{max}, p), \\
& Hier(X_{min}, X_{max}, y, Y_{max}, T_{min}, T_{max}, P - p)), \\
\min_{T_{min} < t < T_{max}} (\max & Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, T_{min}, t, p), \\
& Hier(X_{min}, X_{max}, Y_{min}, Y_{max}, t, T_{max}, P - p)))
\end{aligned}
\tag{1}
$$

(yummy)

Largest block work-overhead tradeoff



(for PollenUS-Hr-Lb)

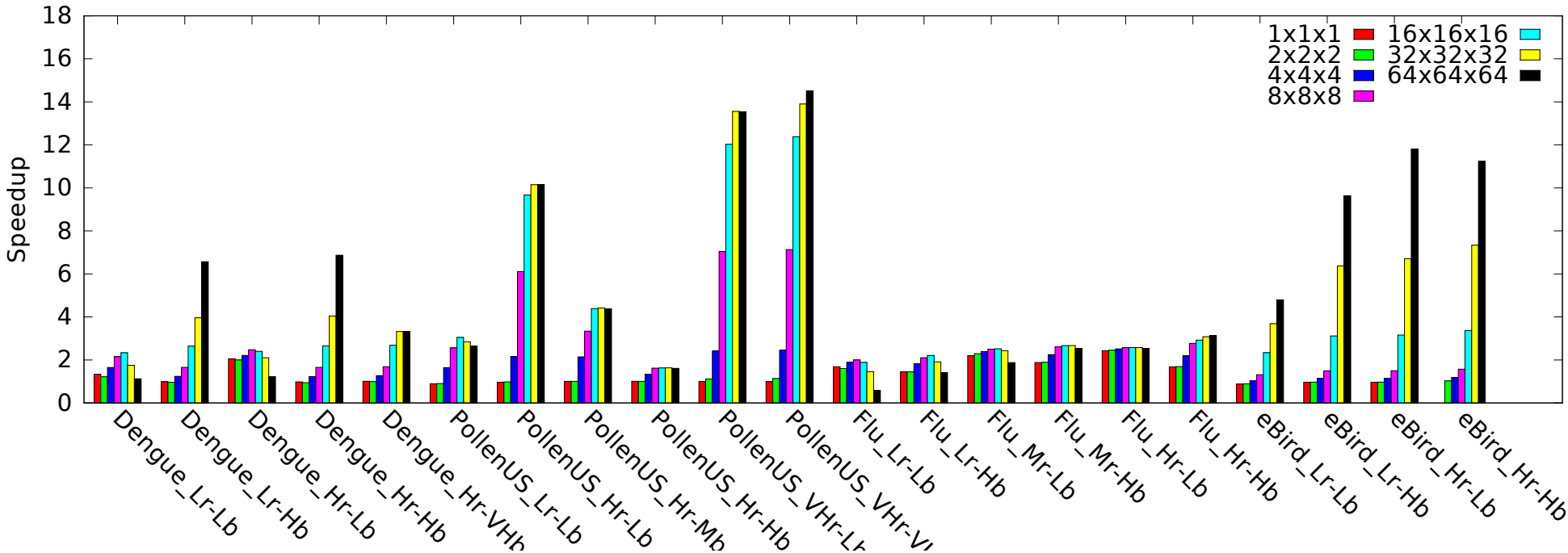
How good are these in practice? Some encouraging preliminary results.

But the partitioning are expensive.

Outline

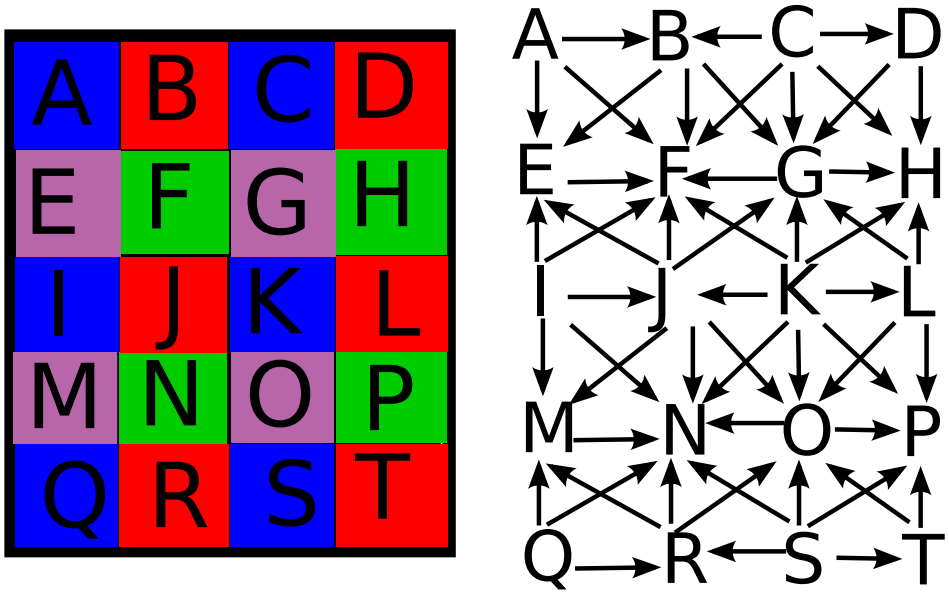
Point decomposition PB-SYM-PD

- Partition the points in a regular AxBxC grid such that each dimension is bigger than the bandwidth.
- For $(a, b, c) \in \{0, 1\}^3$
  - Do in parallel grids $2i + a, 2j + b, 2k + c, \forall i, j, k$
  - Sync
    - No two points closer than twice the bandwidth are processed simultaneously, so no simultaneous cylinder intersection.



Why is this bad? Too many dependencies?

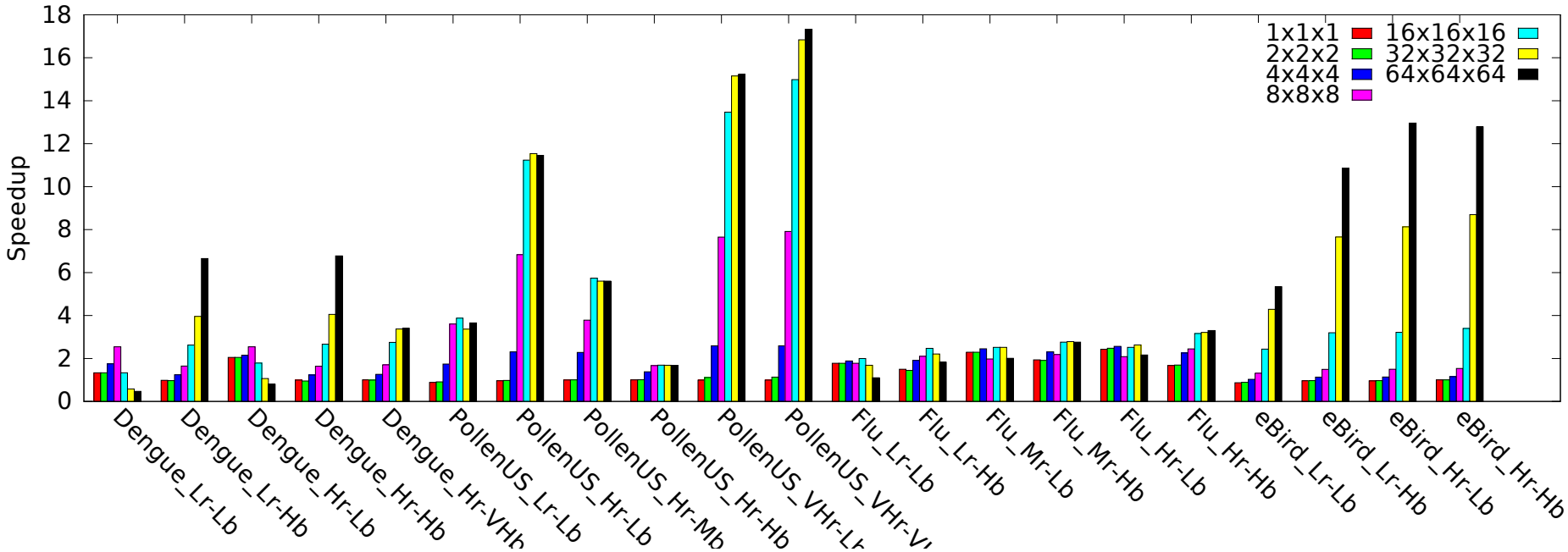
Since all $2i, 2j, 2k$ are done before any $2i+1, 2j, 2k$, there is a forced precedence of $0, 0, 0$ over $3, 0, 0$. But they are not dependent.
This does coloring, instead of doing scheduling.



Building the graph from a coloring is simple (and easily expressed in OpenMP 4.0).

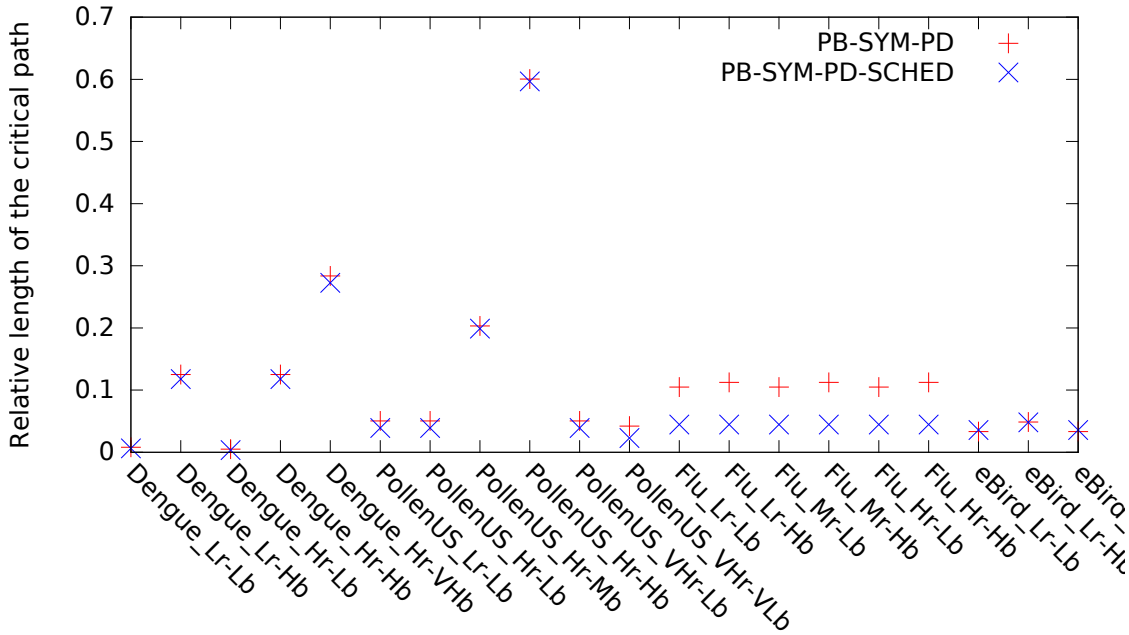
A better coloring with PD-SYM-PD-SCHED

We don't need to color subdomains to minimize the number of colors.
We need a coloring that minimizes the longest chain in the implied graph.
Heuristic: greedily color subdomains in highest number of points first.



(If you don't have a good eye, it is a bit better than before)

Why is it bad? Still too long critical path!

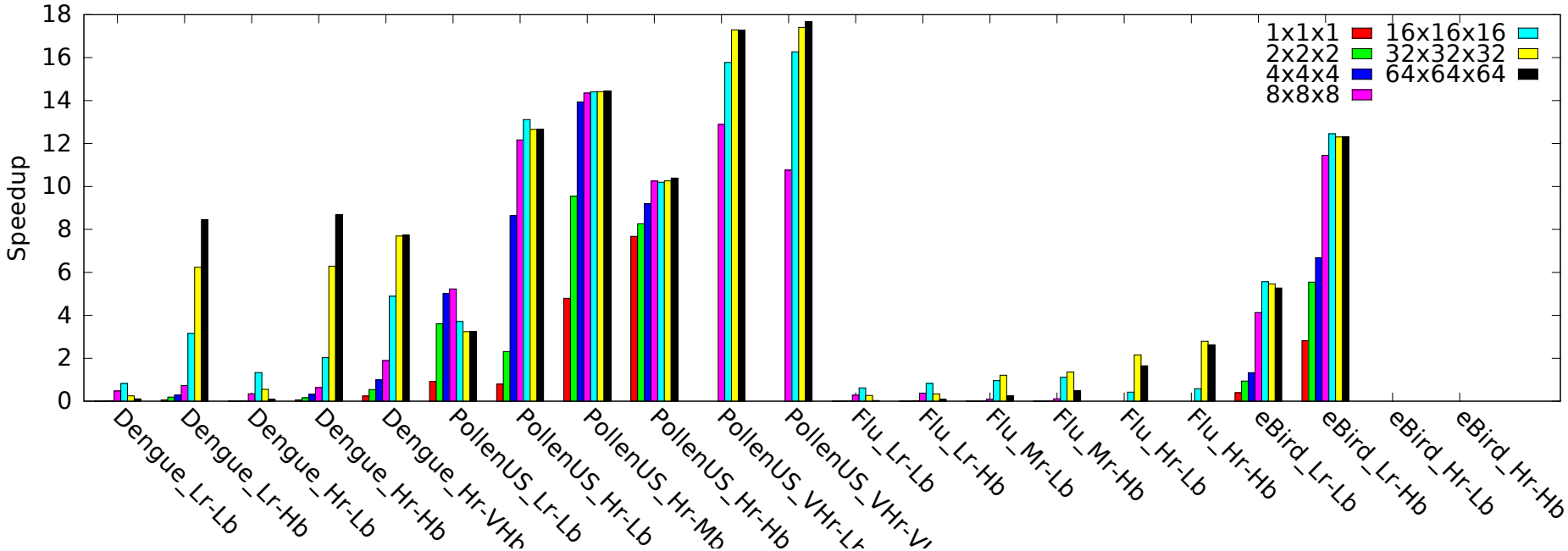


How hard is the coloring/edge-orientation problem to minimize critical path?

- NP-hard in general graph (harder than coloring)
- Trivial on chains
- Polynomial if Bipartite (Julien Hermann made that observation)
- 2D mesh with 5-pt stencil and 3D mesh with 7-pt stencil are bipartite
- On 2D mesh with 9-pt stencil and 3D mesh with 27-pt stencil, one easily builds a 2^{d-1} approximation algorithm. (Based on a 5point stencil, 7point stencil approximation of the graph.)
- On 2D mesh with 9-pt stencil and 3D mesh with 27-pt stencil, NP-Complete? I don't know.
- How good is the heuristic? I don't know

Parallelize long path PB-SYM-PD-SCHED-REP

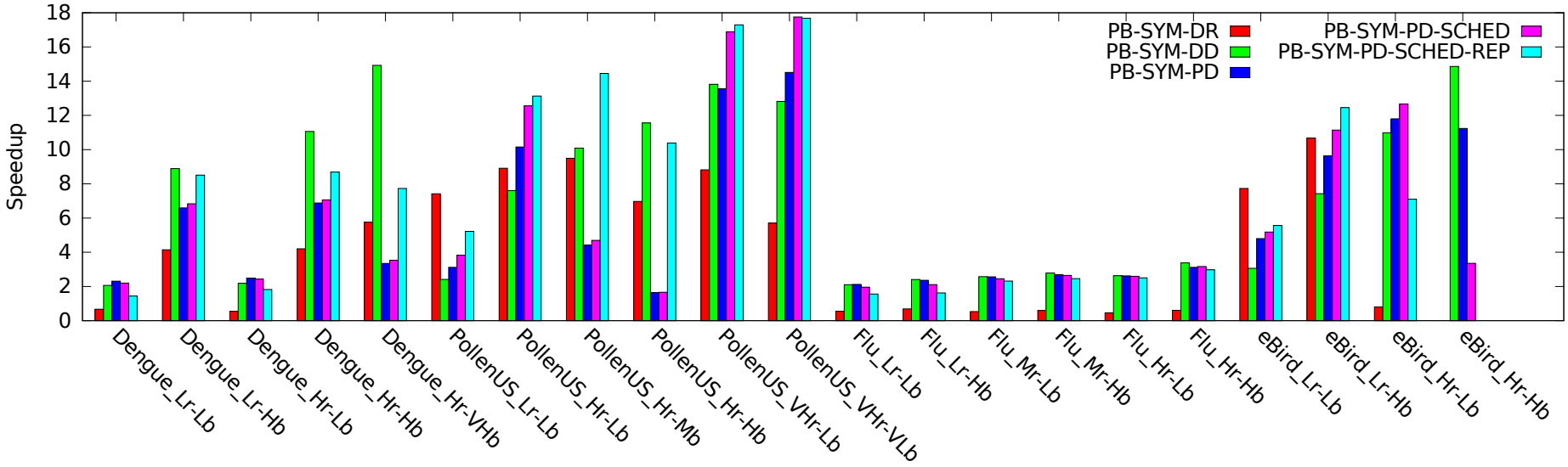
One can replicate a subdomain and get perfect work parallelism. (at the expense of some memory initialization and reduction.)
Heuristic: for all paths longer than $\frac{n}{2P}$, add one copy to all tasks on the path



(This sounds like moldable DAG scheduling.)

Outline

All methods



- Algorithms matter in sequential processing
- The real challenge in parallel processing is often algorithmic

Future Works

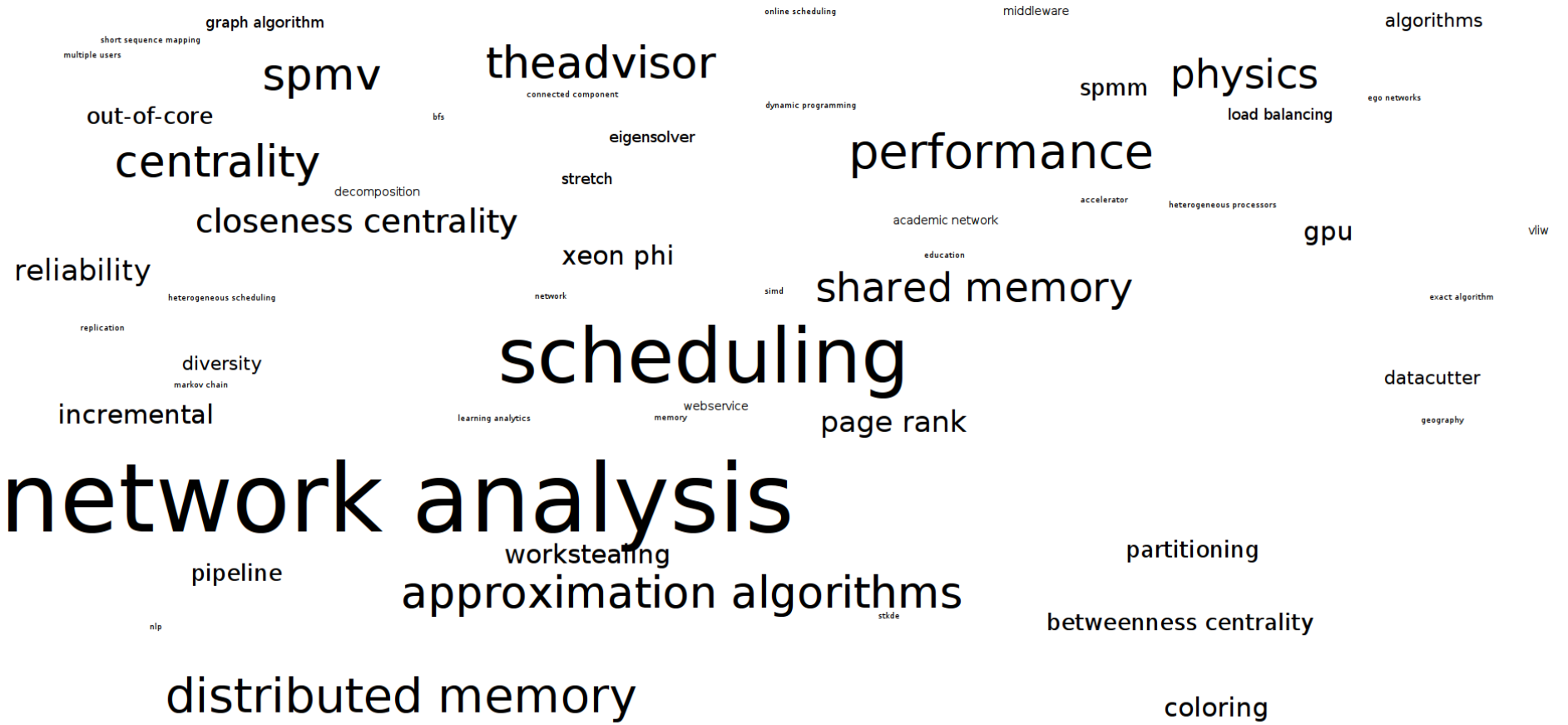
Other platforms:

- GPU (not quite sure how to approach it)
- KNL
- Distributed Memory

Some algorithmic problems:

- Better way to decompose for PB-SYM-DD
- Formally study the edge orientation problem for PB-SYM-DD-SCHED
- Look deeper into the moldable scheduling connection for PB-SYM-PD-SCHED-REP
- Model everything and derive analytical bounds on performance

What else do I do ?



Thank you!

And thanks:

- Dan Janies for pointing out the Flu dataset
- Bora Uçar for pointing out the Gallai-Hasse-Roy-Vitaver theorem
- The US tax payer
  - Support from US NSF XSEDE Supercomputing Resource Allocation (SES170007) is acknowledged.
  - This material is based upon work supported by the National Science Foundation under Grant No. 1652442.

Want to know more?

Read the ICPP 17 paper.

Contact: `esaule@uncc.edu`

Visit: `http://webpages.uncc.edu/~esaule`